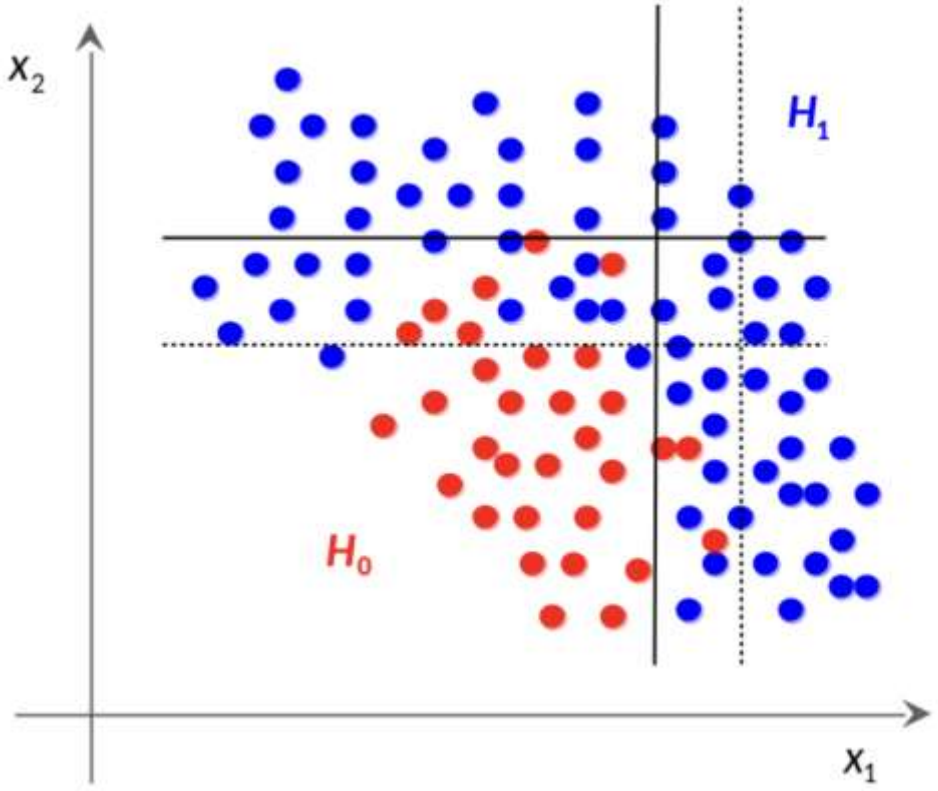
Methods of Artificial Intelligence

Lecture 2

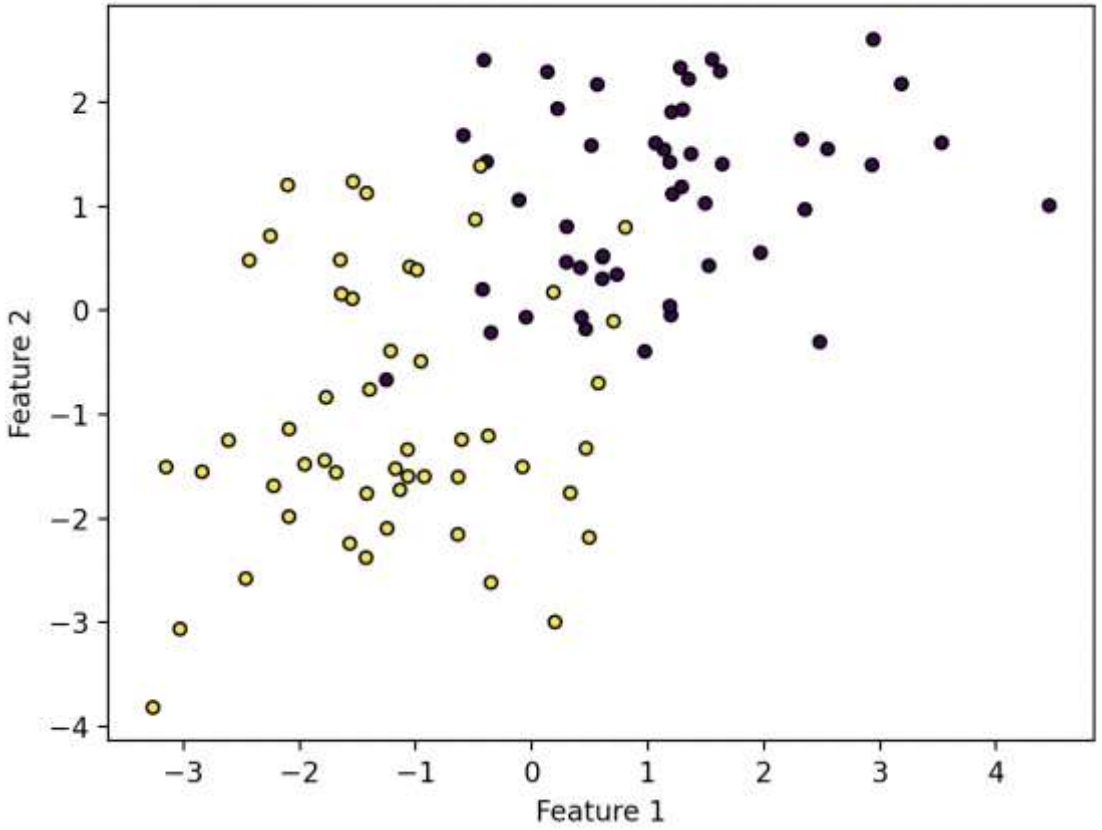
Mateusz Dyndał, WFiIS KOiDC, D11/p113

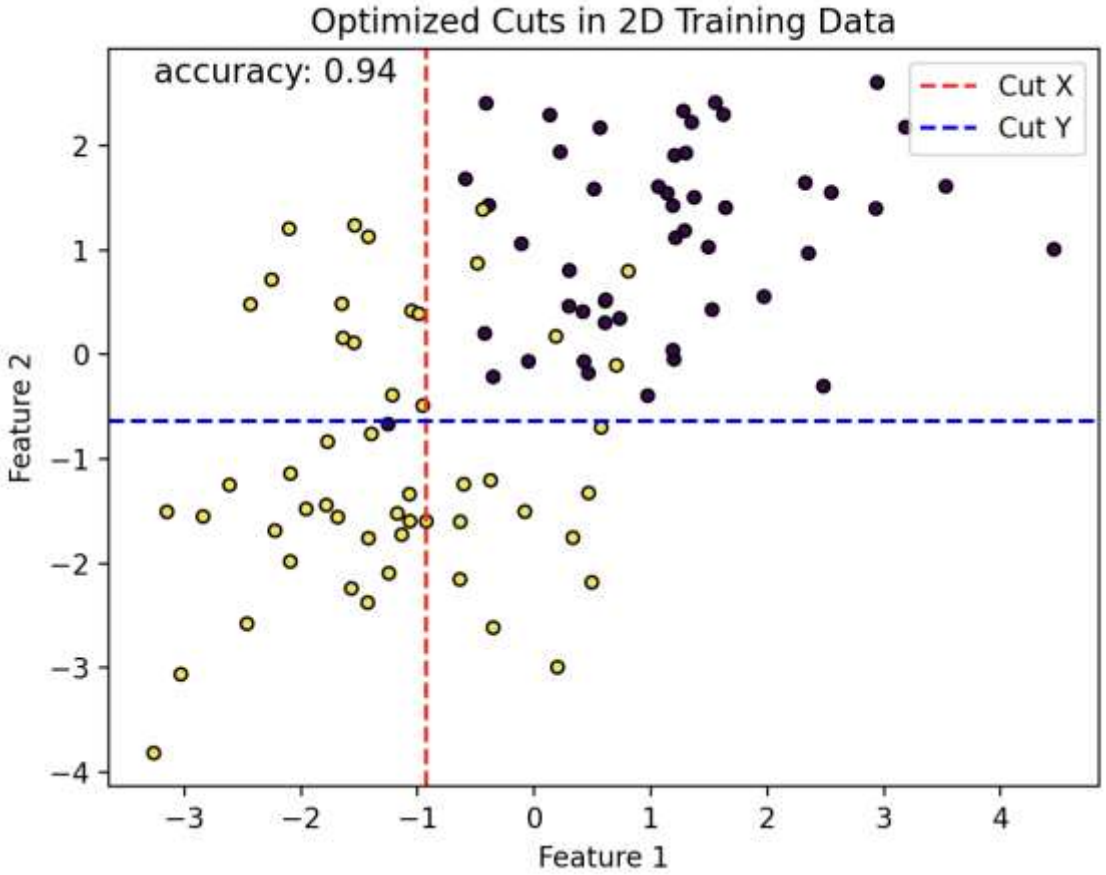
Reminder: selection cuts

- **Optimization of cuts:**
- Move cuts as long as we get the optimal signal vs. background selection. For a given signal efficiency we find the best background rejection → we get the entire ROC curve.
- Optimization methods:
  - Brute force
  - Genetic algorithms
  - Many others...



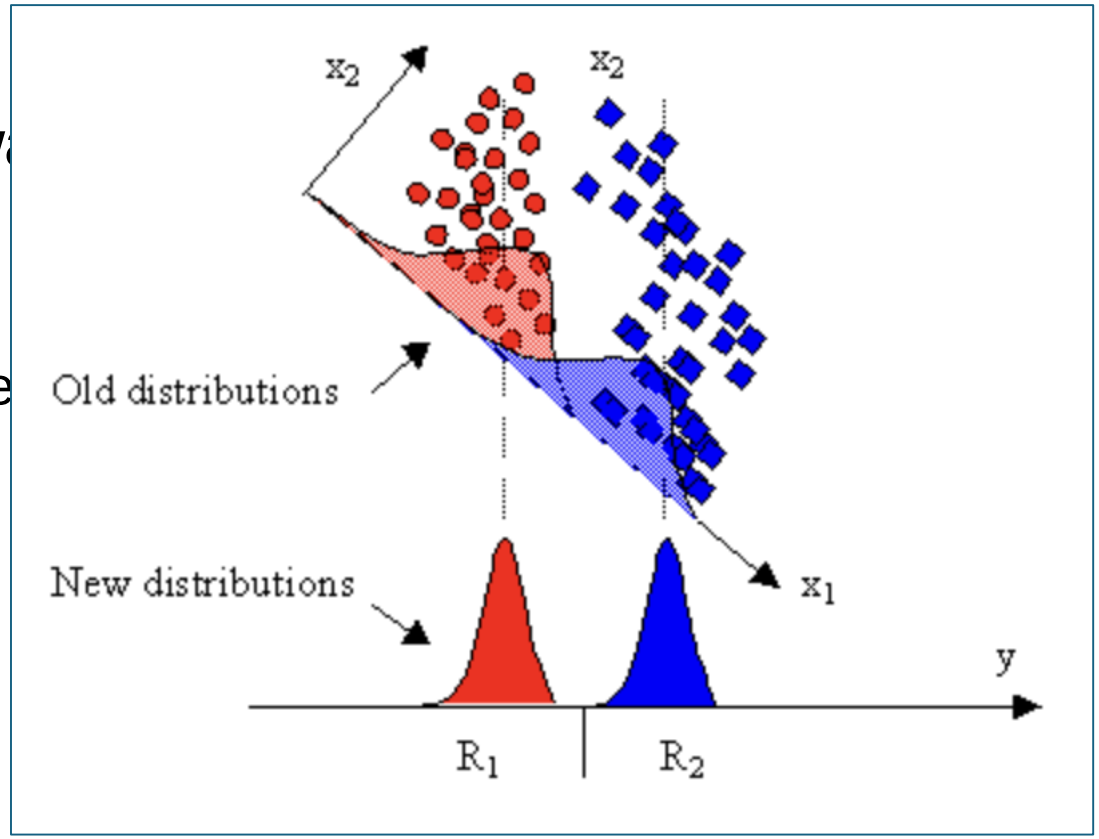
Selection cuts example





Fisher discriminant

- **Procedure:** Projection to one dimension (with maximal separation), then discrimination.
- Equivalent to linear separation.
- We choose a projection vector in such a wa
  that the separation is maximized.
- **Assumptions for new basis:**
  - Maximize distance between projected class me
  - Minimize projected class variance
- **Method introduced by Sir Ronald Aylmer Fisher (1890 – 1962) in 1936.**
- It is an optimal separation for Gaussian distributions.



https://sthalles.github.io/fisher-linear-discriminant/

Fisher Linear Discriminant (FLD)

- FLD selects a projection that maximizes the class separation
  - To do that, it maximizes the ratio between the between-class variance to the within-class variance:

$$J(\boldsymbol{W}) = \frac{(m_2 - m_1)^2}{s_1^2 + s_2^2} \qquad (1)$$

Between-class variance

Within-class variance

W=projection vector

$$s_k^2 = \sum_{n \in C_k} (y_n - m_k)^2 \qquad y_n = \boldsymbol{W}^T x_n \qquad (2)$$

$$J(\boldsymbol{W}) = \frac{\boldsymbol{W}^T S_B \boldsymbol{W}}{\boldsymbol{W}^T S_W \boldsymbol{W}} \qquad (3)$$

$$\boldsymbol{W} \propto S_W^{-1}(m_2 - m_1) \qquad (4)$$

Per-class mean

Within-class variance

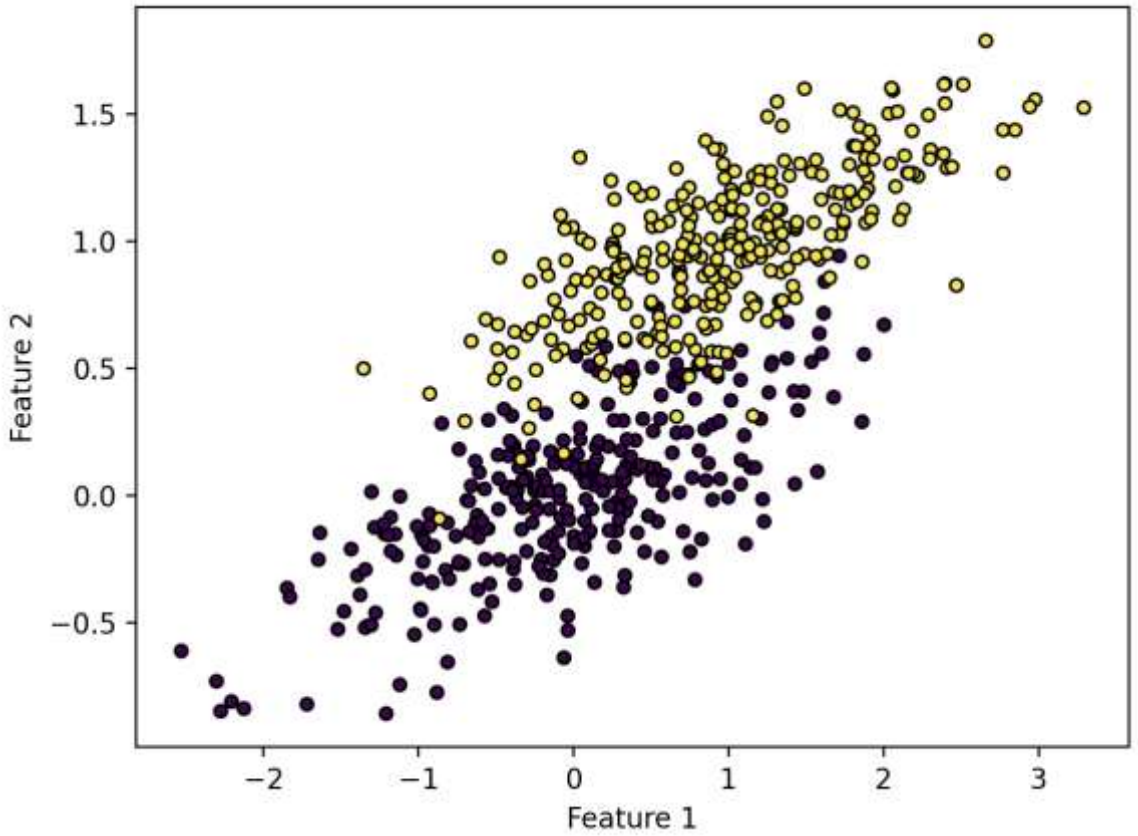
projection equation

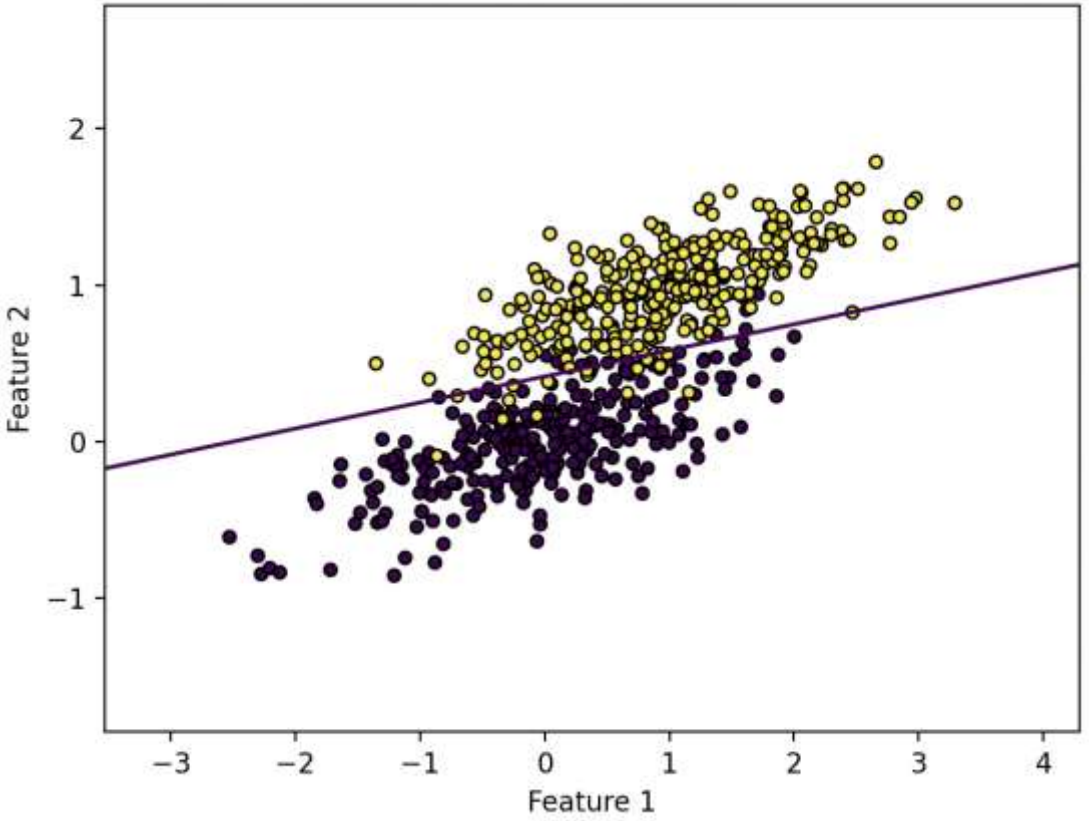
If we substitute the mean vectors m1 and m2 as well as the variance s as given by equations (1) and (2) we get:

$$m_i = \frac{1}{n_i} \sum_{x \in C_i} x$$
$$S_B = (m_2 - m_1)(m_2 - m_1)^T$$
$$S_W = \sum_j^2 \sum_{x \in C_j} (x - m_j)(x - m_j)^T$$

LDA example

```
from sklearn.discriminant_analysis import
LinearDiscriminantAnalysis
clf = LinearDiscriminantAnalysis()
```





Naive Bayes classifier

- Based on the assumption, that variables are independent (so “naive”):
  - Output probability is **a product of probabilities for all variables.**
  - Fast and stable, in general not optimal, but in many cases sufficient.

$$P(y \mid x_1, \ldots, x_n) = \frac{P(y)P(x_1, \ldots x_n \mid y)}{P(x_1, \ldots, x_n)}$$

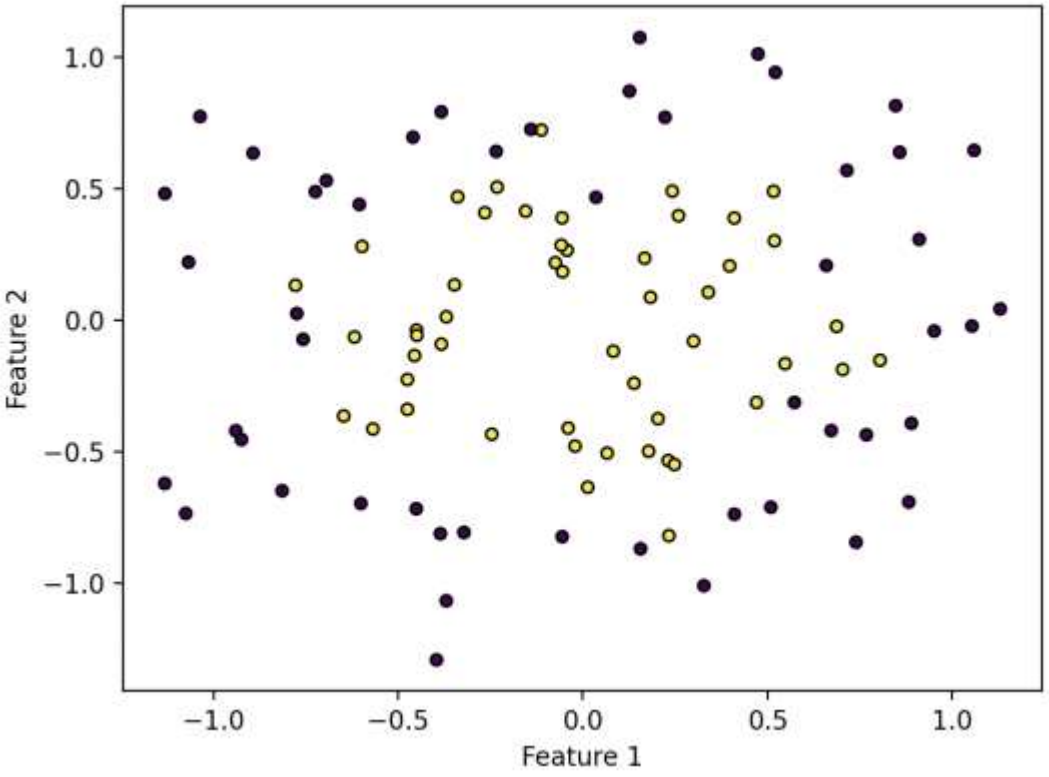
“Naive” assumption:

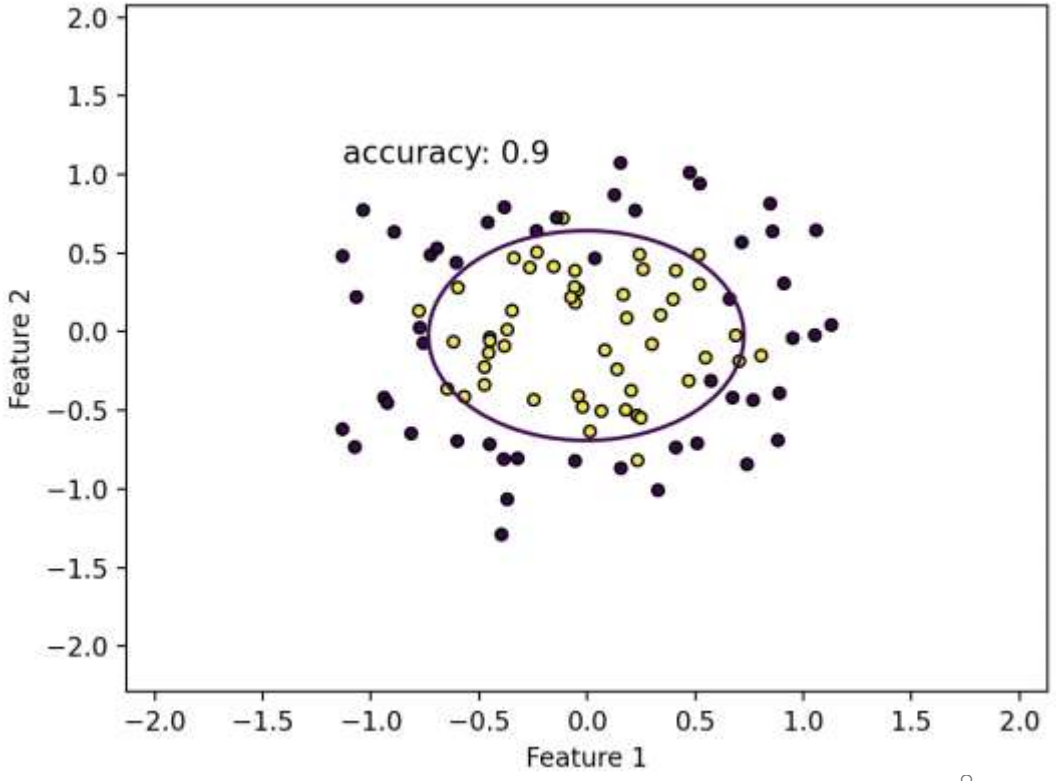
$$P(x_i|y, x_1, \ldots, x_{i-1}, x_{i+1}, \ldots, x_n) = P(x_i|y),$$

$$P(y \mid x_1, \ldots, x_n) = \frac{P(y)\prod_{i=1}^{n} P(x_i \mid y)}{P(x_1, \ldots, x_n)}$$

Naive Bayes example

```
from sklearn.naive_bayes import GaussianNB
gnb = GaussianNB(var_smoothing=0)
gnb.fit(X, y)
```





Short reminder about algebra...

- Let **X** be a p-variate random vector. The **covariance matrix** of **X** is defined as:

$$\Sigma_{XX} = Var(X) = E\{(X-\mu)^T(X-\mu)\}$$
$$= \begin{pmatrix} Var(X_1) & Cov(X_1,X_2) & \dots & Cov(X_1,X_p) \\ Cov(X_2,X_1) & Var(X_2) & \dots & Cov(X_2,X_p) \\ \vdots & \vdots & \ddots & \vdots \\ Cov(X_p,X_1) & Cov(X_p,X_2) & \dots & Var(X_p) \end{pmatrix}.$$

$$Var(X_i) = E\{(X_i-\mu(X_i))\cdot(X_i-\mu(X_i))\}$$
$$Cov(X_i,Y_j) = E\{(X_i-\mu(X_i))\cdot(Y_j-\mu(Y_j))\}$$

Short reminder about algebra...

The correlation matrix refers to the symmetric array of numbers

$$\mathbf{R} = \begin{pmatrix} 1 & r_{12} & r_{13} & \cdots & r_{1p} \\ r_{21} & 1 & r_{23} & \cdots & r_{2p} \\ r_{31} & r_{32} & 1 & \cdots & r_{3p} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ r_{p1} & r_{p2} & r_{p3} & \cdots & 1 \end{pmatrix}$$

where

$$r_{jk} = \frac{s_{jk}}{s_j s_k} = \frac{\sum_{i=1}^{n}(x_{ij} - \bar{x}_j)(x_{ik} - \bar{x}_k)}{\sqrt{\sum_{i=1}^{n}(x_{ij} - \bar{x}_j)^2}\sqrt{\sum_{i=1}^{n}(x_{ik} - \bar{x}_k)^2}}$$

is the Pearson correlation coefficient between variables $\mathbf{x}_j$ and $\mathbf{x}_k$.

Short reminder about algebra...

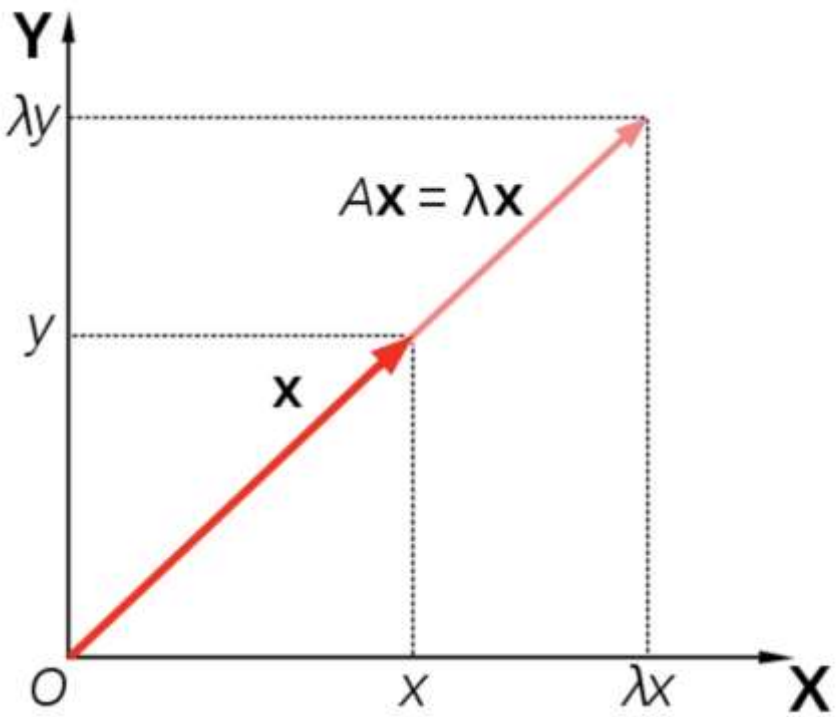
- **Eigenvalues and eigenvectors**
- In essence, an eigenvector v of a linear transformation T is a non-zero vector that, when T is applied to it, does not change direction.
- Applying **T** to the eigenvector only scales the eigenvector by the scalar value λ, called an eigenvalue. This condition can be written as a following equation:

$$\mathbf{T}\vec{v} = \lambda\vec{v}$$

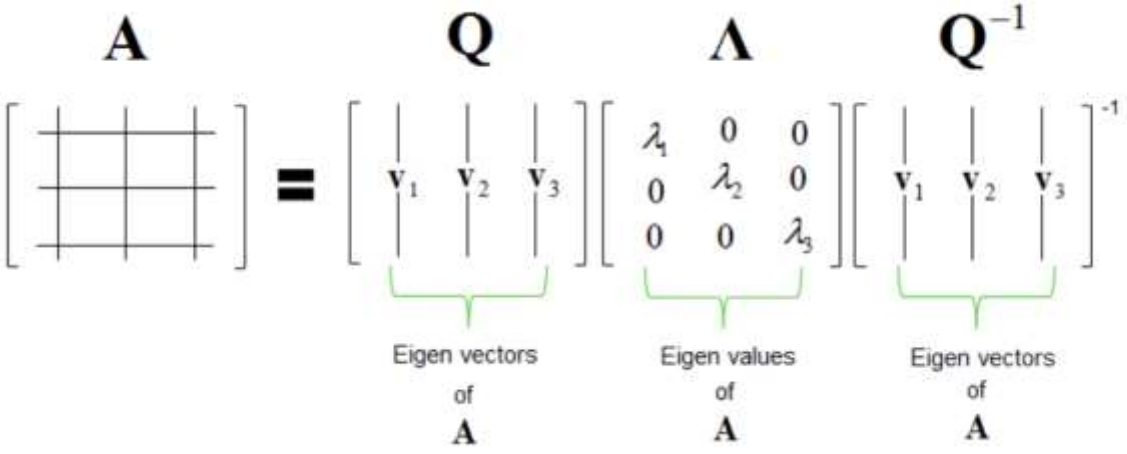
- referred to as the eigenvalue equation or eigenequation.
- In general, λ may be any scalar. For example, λ may be negative, in which case the eigenvector reverses direction as part of the scaling, or it may be zero or complex.

Short reminder about algebra...

- **Eigenvalues and eigenvectors**
- Matrix **A** acts by stretching the vector x, not changing its direction, so x is an eigenvector of **A**:



Eigen-decomposition of a matrix:



Short reminder about algebra...

- **How to find eigenvalues and eigenvectors?**

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$$

then v is an eigenvector of the linear transformation **A** and the scale factor λ is the eigenvalue corresponding to that eigenvector. It is the eigenvalue equation for the matrix **A**.

Equation can be stated equivalently as:

$$(\mathbf{A} - \lambda\mathbf{I})\mathbf{v} = \mathbf{0}$$

where **I** is the n by n identity matrix and 0 is the zero vector. Values of λ are therefore the solutions of the equation:

$$det\,(\mathbf{A} - \lambda\mathbf{I}) = 0$$

and the eigenvectors can be found by solving $(\mathbf{A} - \lambda\mathbf{I})\mathbf{v} = \mathbf{0}$.

Principal Components Analysis (PCA)

- **PCA** is a statistical method for reducing a cases-by-variables data table to its essential features, called **principal components**.
- Principal components are a few linear combinations of the original variables that maximally explain the variance of all the variables.
- A principal component analysis (PCA) plot shows similarities between groups of samples in a data set. Each point on a PCA plot represents a correlation between an initial variable and the first and second principal components.
- PCA reduces the number of variables or features in a data set while still preserving the most important information like major trends or patterns. This reduction can decrease the time needed to train a machine learning model and helps avoid overfitting in a model.
- PCA is **used in unsupervised learning & dimensionality reduction**
- **In particular, PCA is used for:**
  - Data Visualization
  - Data Reduction
  - Data Classification
  - Noise Reduction

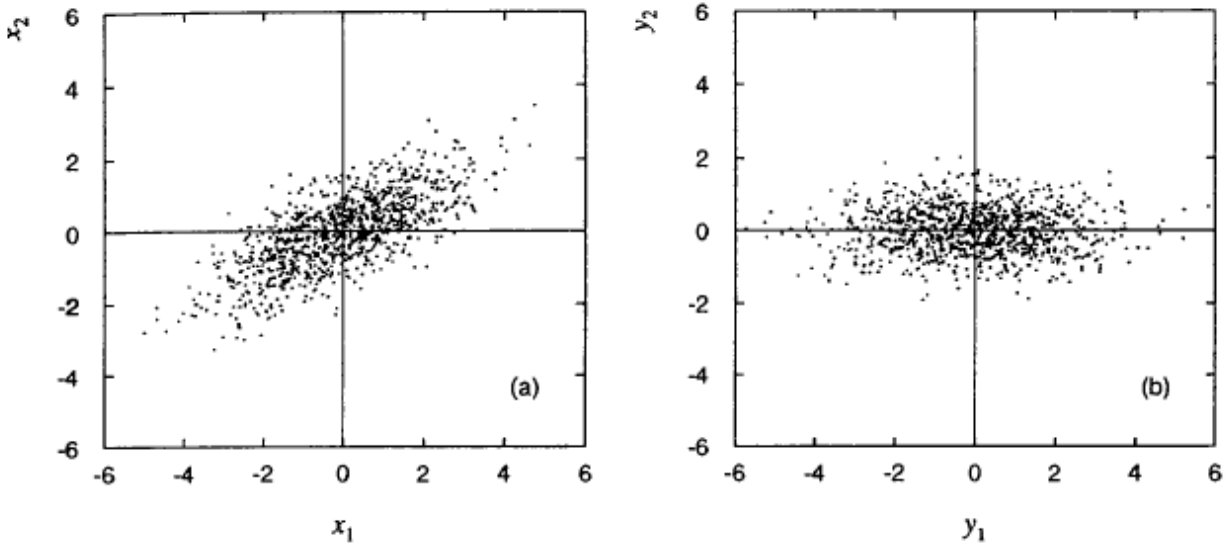
Steven M. Holland, Univ. of Georgia]: Principal Components Analysis
skymind.ai: Eigenvectors, Eigenvalues, PCA, Covariance and Entropy
Lindsay I. Smith: A tutorial on Principal Component Analysis
Nat Rev Methods Primers **2**, 100 (2022)

Principal Components Analysis (PCA)

- For two dimensions this is a simple calculation, for more we just use computer programs. In the case of 2d it can be shown:

$$\mathcal{C} = \begin{pmatrix} \sigma_1^2 & \rho\sigma_1\sigma_2 \\ \rho\sigma_1\sigma_2 & \sigma_1^2 \end{pmatrix} \qquad \vec{\lambda}^{(1)} = \begin{pmatrix} cos\theta \\ sin\theta \end{pmatrix} \qquad \vec{\lambda}^{(2)} = \begin{pmatrix} -sin\theta \\ cos\theta \end{pmatrix}$$

$$\theta = \frac{1}{2} tg^{-1}\left(\frac{2\rho\sigma_1\sigma_2}{\sigma_1^2 - \sigma_2^2}\right)$$



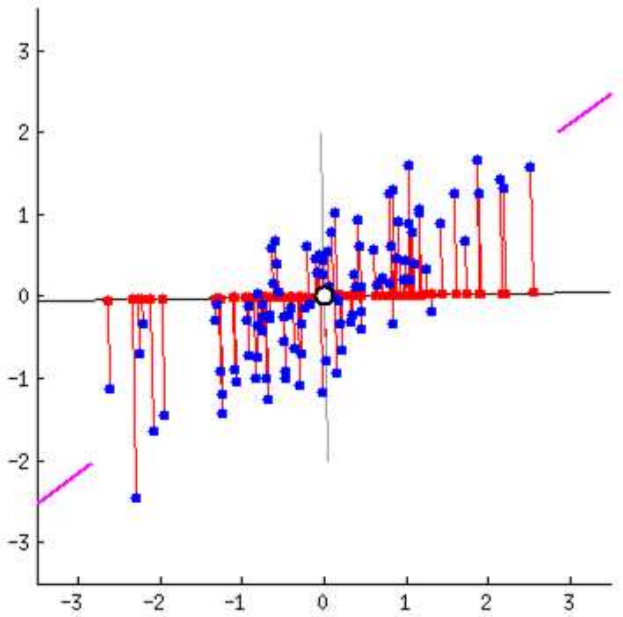
PCA: step-by-step

Construction of Principal Components:

1) The first principal component accounts for the **largest possible variance** in the data set.

2) The second principal component - should be uncorrelated with (i.e., perpendicular to) the first principal component and that it accounts for the next highest variance.

3) This continues until a total of *p* principal components have been calculated, equal to the original number of variables.



https://builtin.com/data-science/step-step-explanation-principal-component-analysis

PCA: step-by-step

Step 1: Standardization

- The aim of this step is to standardize the range of the continuous initial variables so that each one of them contributes equally to the analysis.
- More specifically, the reason why it is critical to perform standardization prior to PCA, is that the latter is quite sensitive regarding the variances of the initial variables.
- That is, if there are large differences between the ranges of initial variables, those variables with larger ranges will dominate over those with small ranges (for example, a variable that ranges between 0 and 100 will dominate over a variable that ranges between 0 and 1), which will lead to biased results.
- So, transforming the data to comparable scales can prevent this problem.
- Mathematically, this can be done by subtracting the mean and dividing by the standard deviation for each value of each variable.

- Now, all variables are at the same scale

PCA: step-by-step

Step 2: Covariance Matrix Computation

- The aim of this step is to understand how the variables of the input data set are varying from the mean with respect to each other, or in other words, to see if there is any relationship between them. Because sometimes, variables are highly correlated in such a way that they contain redundant information. So, in order to identify these correlations, we compute the covariance matrix.
- The covariance matrix is a $p \times p$ symmetric matrix (where p is the number of dimensions) that has as entries the covariances associated with all possible pairs of the initial variables. For example, for a 3-dimensional data set with 3 variables x, y, and z, the covariance matrix is a 3×3 data matrix of this from:

$$\begin{bmatrix} Cov(x,x) & Cov(x,y) & Cov(x,z) \\ Cov(y,x) & Cov(y,y) & Cov(y,z) \\ Cov(z,x) & Cov(z,y) & Cov(z,z) \end{bmatrix}$$

- Note: Sign of the covariance matrix means that the variables can be positively or negatively correlated

PCA: step-by-step

Step 3: Compute the eigenvectors and eigenvalues of the covariance matrix to identify the principal components

- Eigenvectors and eigenvalues are computed from the covariance matrix in order to determine the ***principal components*** of the data.
- By ranking your eigenvectors in order of their eigenvalues, highest to lowest, you get the principal components in order of significance.

Example: data set is 2-dimensional with 2 variables $(\boldsymbol{X}, \boldsymbol{Y})$ and that the eigenvectors and eigenvalues:

$$v1 = \begin{bmatrix} 0.6778736 \\ 0.7351785 \end{bmatrix} \qquad \lambda_1 = 1.284028$$

$$v2 = \begin{bmatrix} -0.7351785 \\ 0.6778736 \end{bmatrix} \qquad \lambda_2 = 0.04908323$$

$\boldsymbol{\lambda_1 > \lambda_2}$

so: $PC1$ is an eigenvector v_1, $PC2 - v_2$.

- ✓ which one is more important? Let's divide the eigenvalue of each component by the sum of eigenvalues and we get percentage of variance (information) accounted for by each component
- ✓ here - $PC1$ and $PC2$ carry respectively 96 percent and 4 percent of the variance of the data.

PCA: step-by-step

Step 4: Create a Feature Vector

- We need to choose whether to keep all these components or discard those of lesser significance (of low eigenvalues), and form with the remaining ones a matrix of vectors that we call ***Feature vector***.
- Feature vector is simply a matrix that has as columns the eigenvectors of the components that we decide to keep.
- This makes it the first step towards dimensionality reduction, because if we choose to keep only ***p*** eigenvectors (components) out of ***n***, the final data set will have only ***p*** dimensions.

Ex. from previous slide – feature vector takes v_1 and v_2 :

$$\begin{bmatrix} 0.6778736 & -0.7351785 \\ 0.7351785 & 0.6778736 \end{bmatrix}$$

we can can discard v_2 take the v_1 only (loss of information?):

$$\begin{bmatrix} 0.6778736 \\ 0.7351785 \end{bmatrix}$$

v_2 was carrying only 4 percent of the information, the loss will be therefore not important and we will still have 96 percent of the information that is carried by v_1.

PCA: step-by-step

We have not changed the data yet – we performed some calculation and decide we selected the principal componenent(s).

Step 5: Recast the Data Along the Principal Components Axes

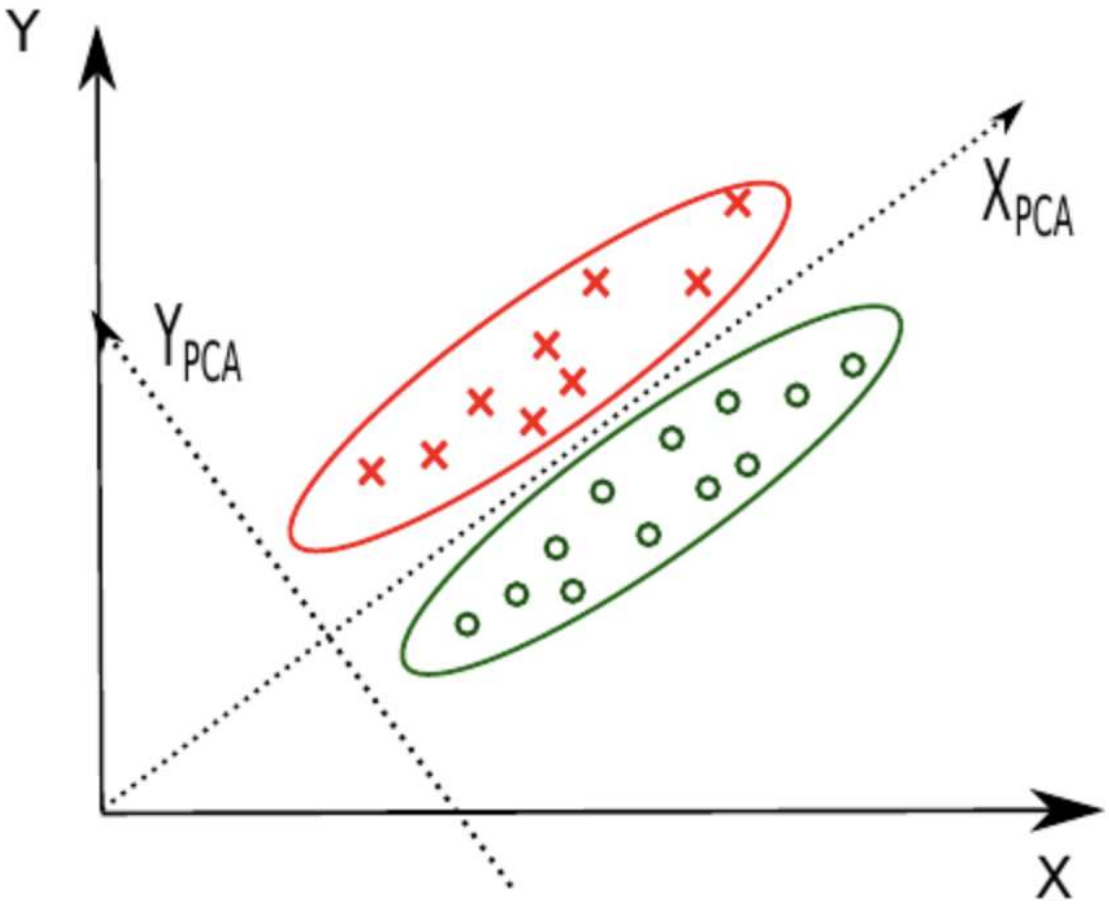
- The aim is to use the feature vector formed using the eigenvectors of the covariance matrix, to reorient the data from the original axes to the ones represented by the principal components (hence the name Principal Components Analysis).
- This can be done by multiplying the transpose of the original data set by the transpose of the feature vector.

$$Final\ Data\ Set = (Feature\ Vector)^T\ (Standarised\ Original\ Data\ Set)^T$$

https://youtu.be/FD4DeN81ODY

When PCA might fail?

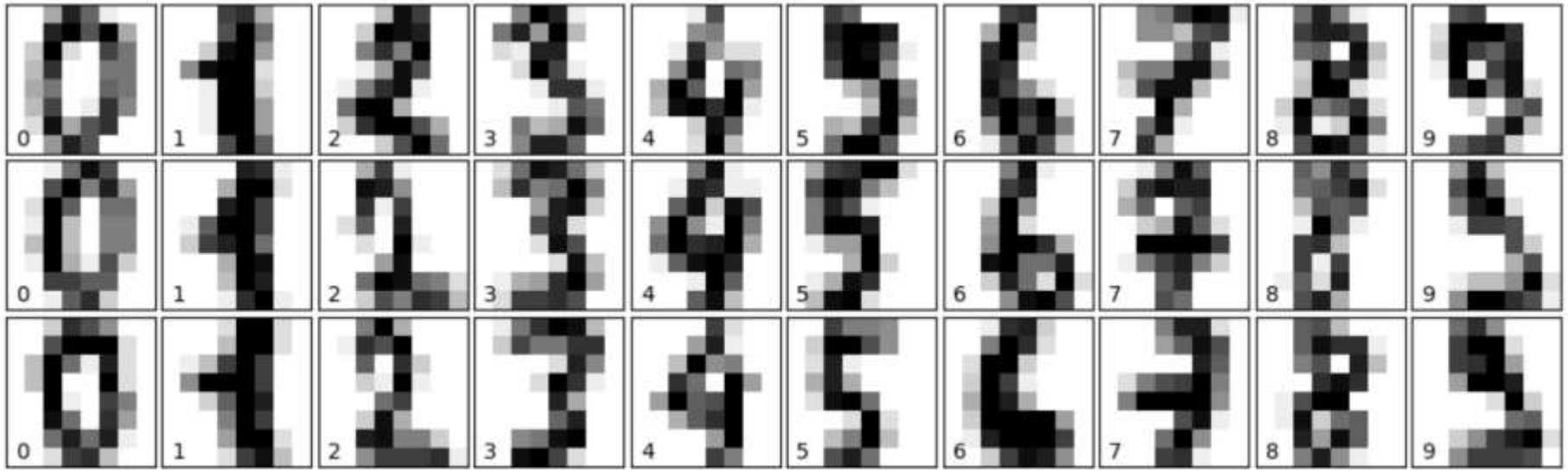
- When events are separable by a simple cut, PCA doesn't help...



PCA example (hand-written digits)

- The program:
  - Reads handwritten digits
  - Performs PCA
  - Displays two first principal components:
  - Classification using Naive Bayes and LDA
- Each event is an 8x8 image

```
from sklearn.datasets import
load_digits
digits = load_digits()
```

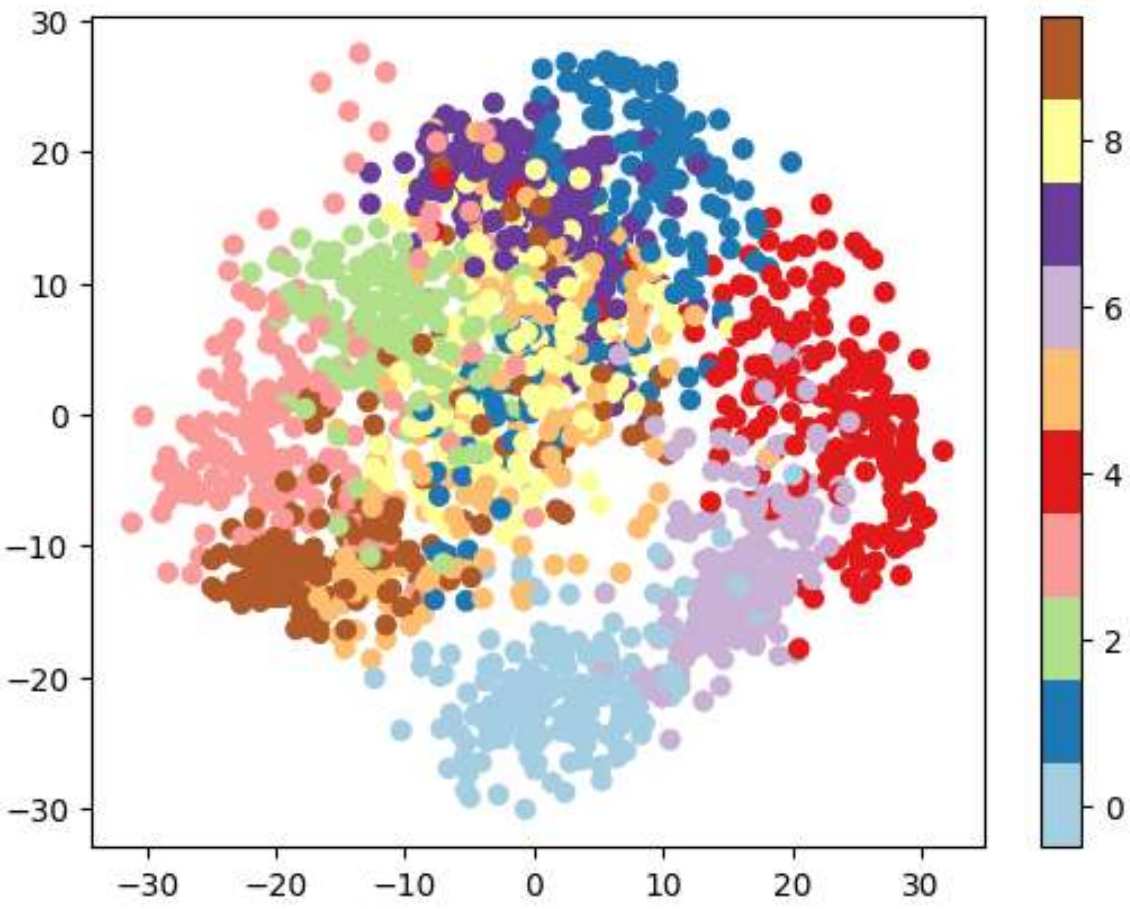


PCA example (hand-written digits)

- **Plotting a projection on the 2 first principal axis**

```
from sklearn.decomposition import PCA
pca = PCA(n_components=2)
proj = pca.fit_transform(digits.data)
print(proj.shape)
```

```
(1797, 2)
```

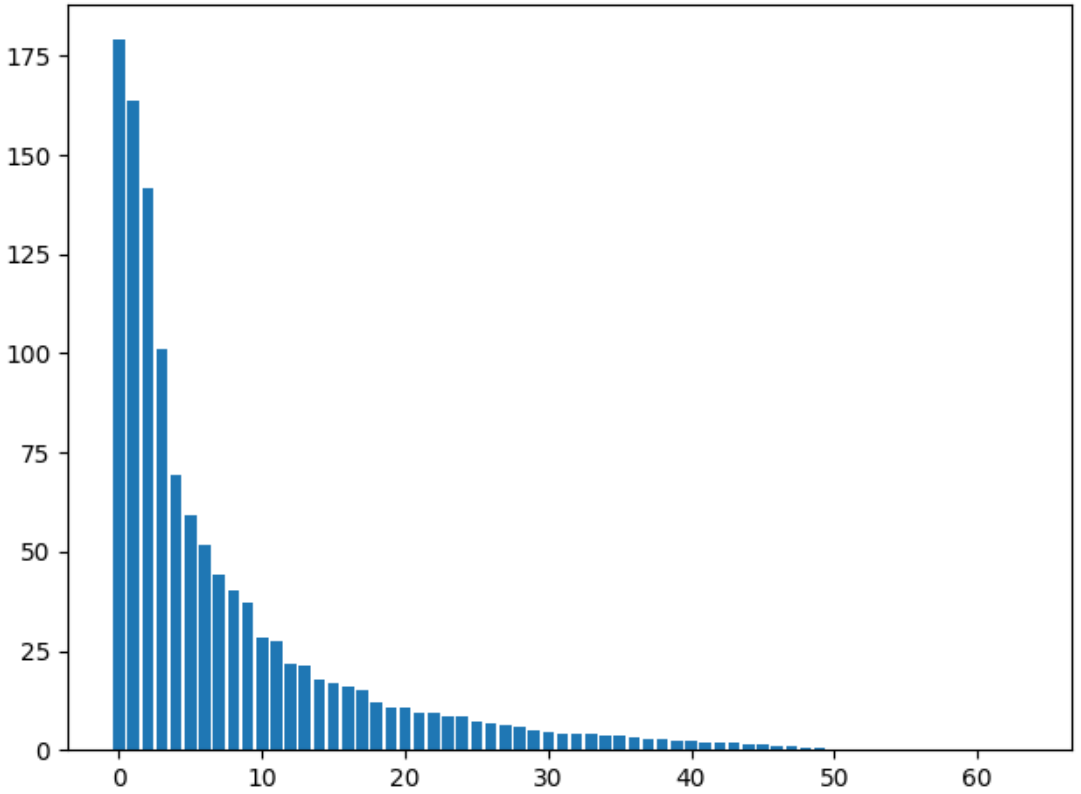



```
#Plot the projection
plt.figure()
plt.scatter(proj[:, 0], proj[:, 1], c=digits.target.astype(int), cmap=plt.get_cmap('Paired', 10), vmin=-0.5, vmax=9.5)
plt.colorbar()
```

PCA example (hand-written digits)

- **Plot principal components**

```
# We use now 64 principal components
pca = PCA(n_components=64)

proj = pca.fit_transform(digits.data)
```

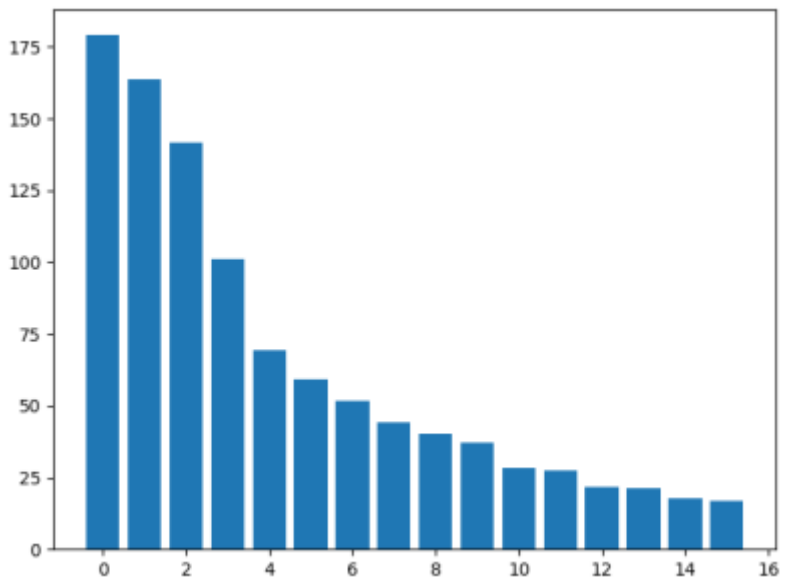



```
principal = pca.explained_variance_
print(pca.explained_variance_)

fig, ax = plt.subplots(tight_layout=True)
ax.bar( [x for x in range(principal.size)], principal)
```

PCA example (hand-written digits)

- We set n=16

```
# Number of components after PCA
n_components=16

pca = PCA(n_components)

proj = pca.fit_transform(digits.data)
```



- Split data into train and test datasets:

```
import numpy as np
from sklearn.model_selection import train_test_split

# Instead of 8x8 digits we have PCA transformed vectors of the length n_components
# split the data into training and validation sets
seed = np.random.randint(0,1000)
X_train, X_test, y_train, y_test = train_test_split(pca.fit_transform(digits.data), digits.target, random_state=seed)
```

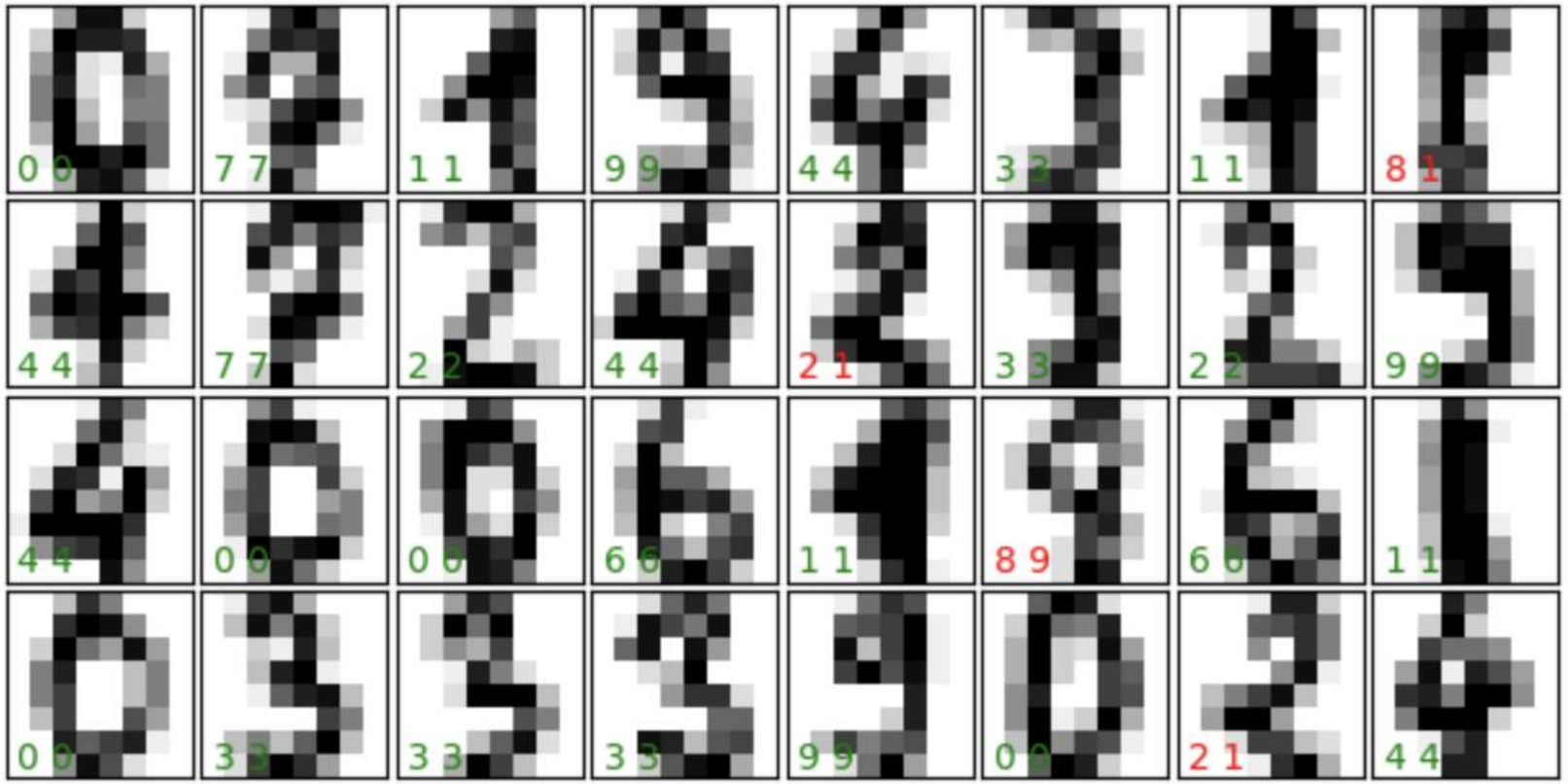
PCA example (hand-written digits)

- Classify with Naive Bayes Classifies / Fisher Linear Discriminants:

```
from sklearn.naive_bayes import GaussianNB
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis


# train the model
# comment out the model you don't use
clf = GaussianNB()
#clf = LinearDiscriminantAnalysis()


print(clf.fit(X_train, y_train))
#print(clf.feature_importances_)

# use the model to predict the labels of the test data
# test data were not used for training!
predicted = clf.predict(X_test)
expected = y_test


print("Score = ",clf.score(X_test, y_test))
```

PCA example (hand-written digits)

- Plotting the predictions




```
In the plot above first label is the expected (true) digit,
second the reconstructed digit
```

PCA example (hand-written digits)

- Number of matches

```
[ ] matches = (predicted == expected)
    print(matches.sum())
```

```
417
```

The total number of data points

```
[ ] print(len(matches))
```

```
450
```

And now, the ratio of correct predictions

```
matches.sum() / float(len(matches))
```

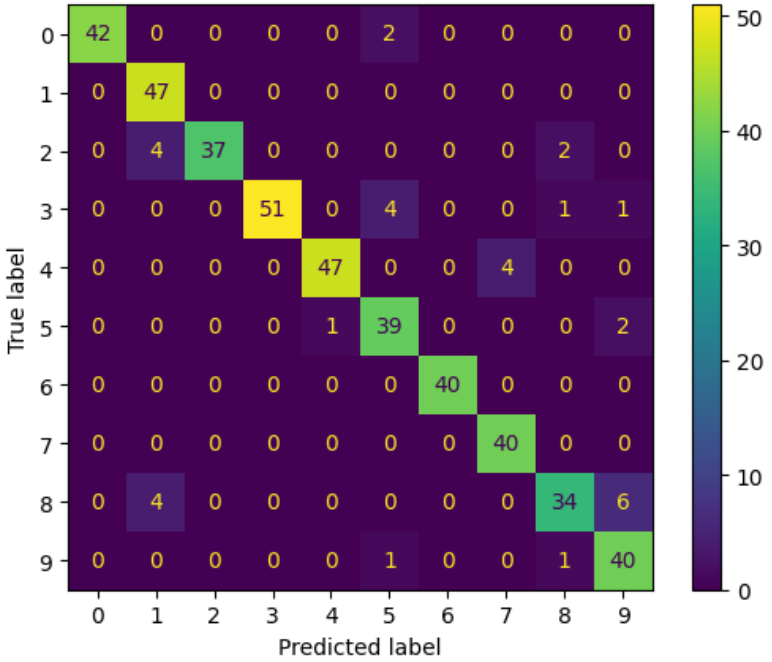
```
0.9266666666666666
```

PCA example (hand-written digits)

- Confusion matrix (expected true label vs. the reconstructed label)

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

cm = confusion_matrix(expected, predicted)
print(cm)
```

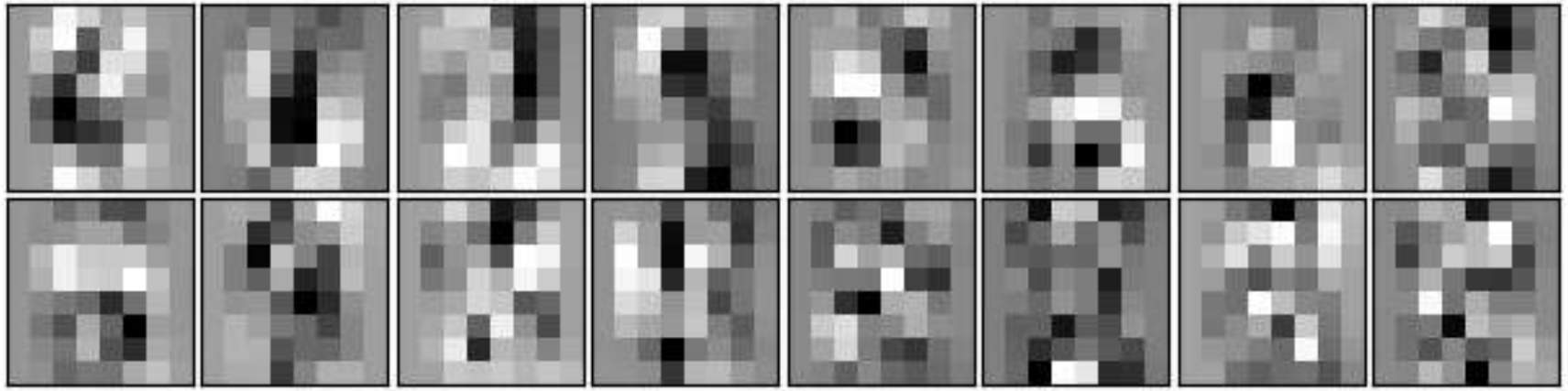
```
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot()
plt.show()
```



PCA example (hand-written digits)

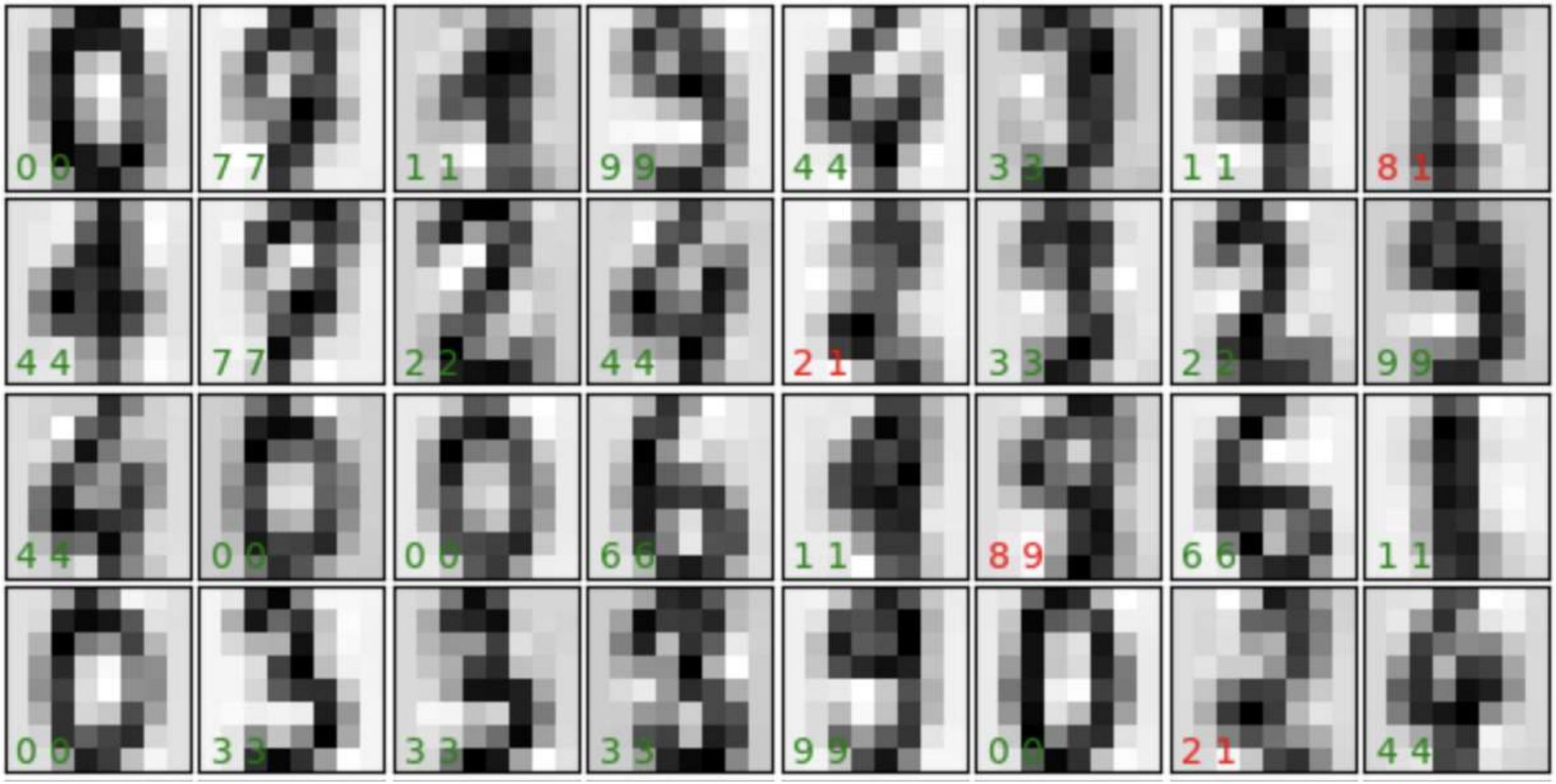
- Plotting eigen-digits (the eigen-vectors of our analysis)

```
eigendigits = pca.components_.reshape((n_components, 8, 8))

# Plot the prediction
fig = plt.figure(figsize=(6, 6))  # figure size in inches
fig.subplots_adjust(left=0, right=1, bottom=0, top=1, hspace=0.05, wspace=0.05)

# plot the digits: each image is 8x8 pixels
for i in range(len(eigendigits)):
    ax = fig.add_subplot(8, 8, i + 1, xticks=[], yticks=[])
    ax.imshow(eigendigits[i], cmap=plt.cm.binary,
              interpolation='nearest')
```



PCA example (hand-written digits)

- Plotting digits after n=16 PCA



Digression on decorrelation

- **Removing correlation between the variables -> rotation in the space of variables**
- **How to**

Covariance matrix:

$$\begin{bmatrix} Var(x_1) & \dots & Cov(x_1, x_n) \\ \vdots & \ddots & \vdots \\ Cov(x_n, x_1) & \dots & Var(x_n) \end{bmatrix}$$

where:

- Sample Variance: $var(x) = \frac{\sum_1^n (x_i - \bar{x})^2}{n-1}$
- Sample Covariance: $cov(x, y) = \frac{\sum_1^n (x_i - \bar{x})(y_i - \bar{y})}{n-1}$
- $\bar{x} =$ mean of sample data.
- $n =$ number of observations in the dataset.
- $x_i =$ observations in dataset x.

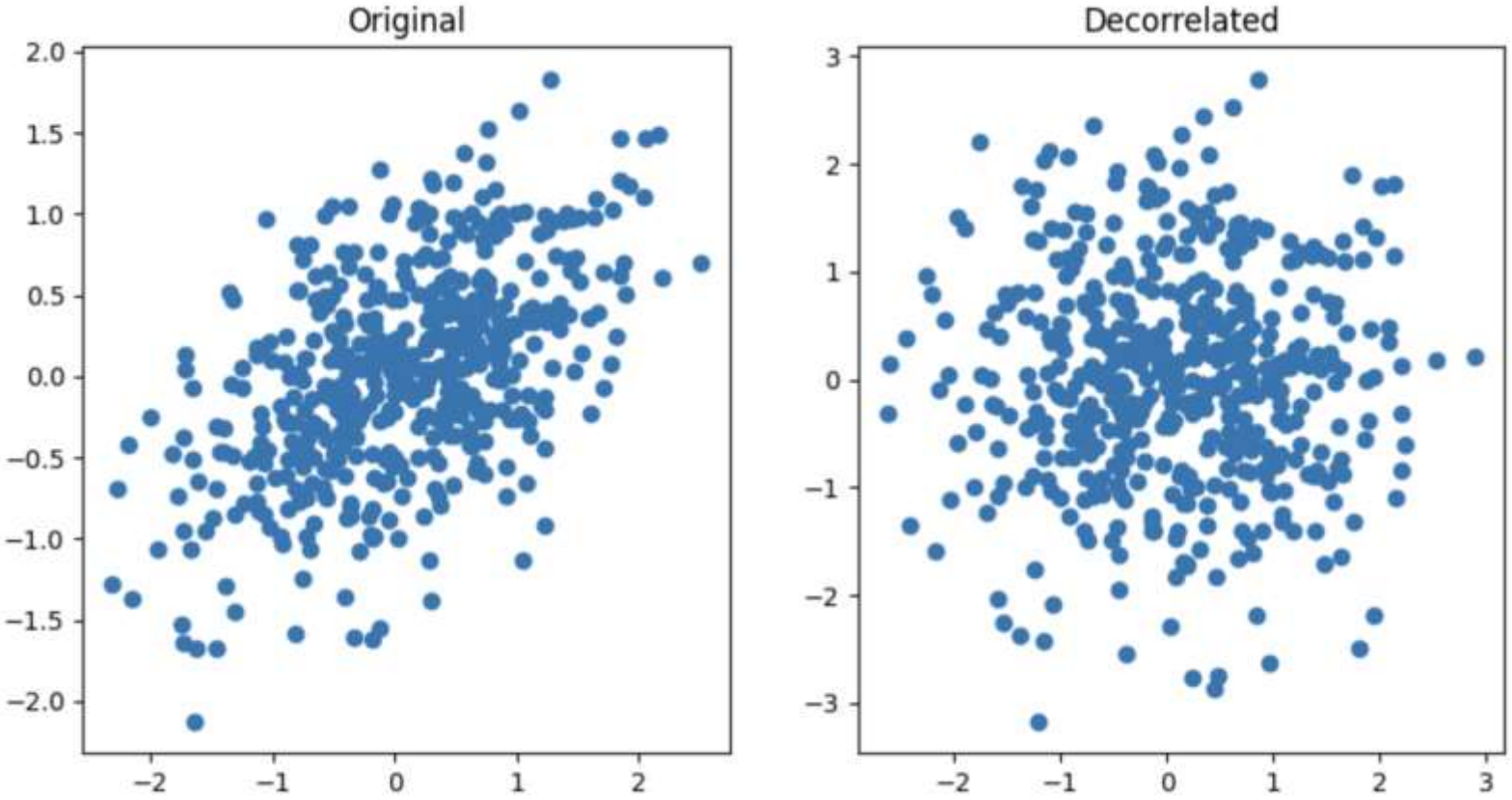
The decorrelated dataset x is given by:

$$x \longrightarrow (C')^{-1} x$$

where C' is the square-root of covariance matrix C. It is a matrix that multiplied with itself yields C : $C = (C' * C')$.

Digression on decorrelation

- **Can compute the square-root matrix by using e.g. the *scipy.linalg.sqrtm()* function and then invert it using *np.linalg.inv()***

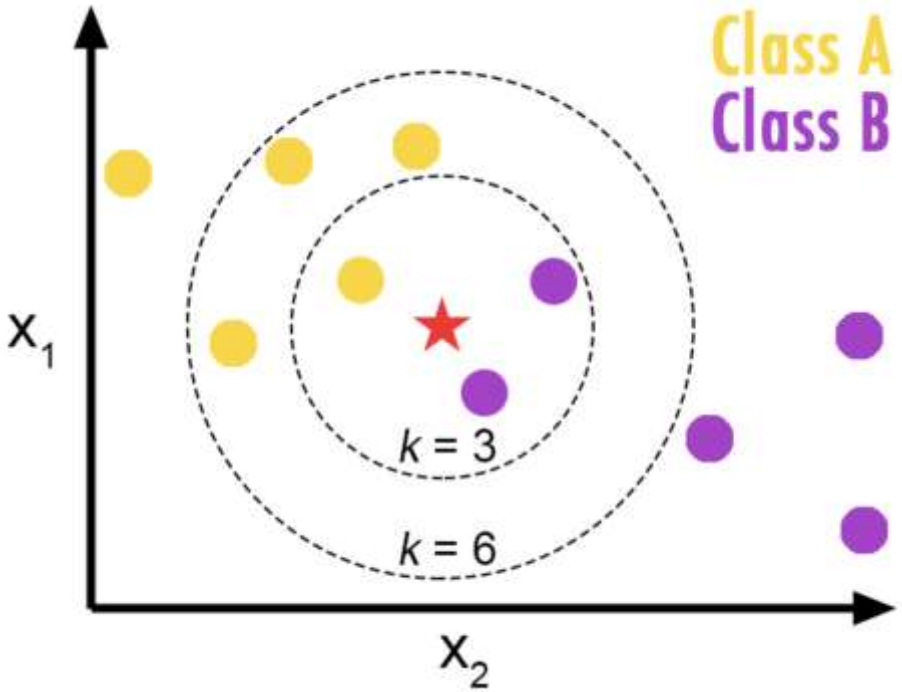


k-nearest neighbours classifier (KNN)

- Proposed in 1951
- An event is qualified to the class, to which belongs the majority of it's k nearest neighbours, or the probability of belonging to the class "signal" is calculated as:

$$p(S|x) = \frac{k_S}{k}$$

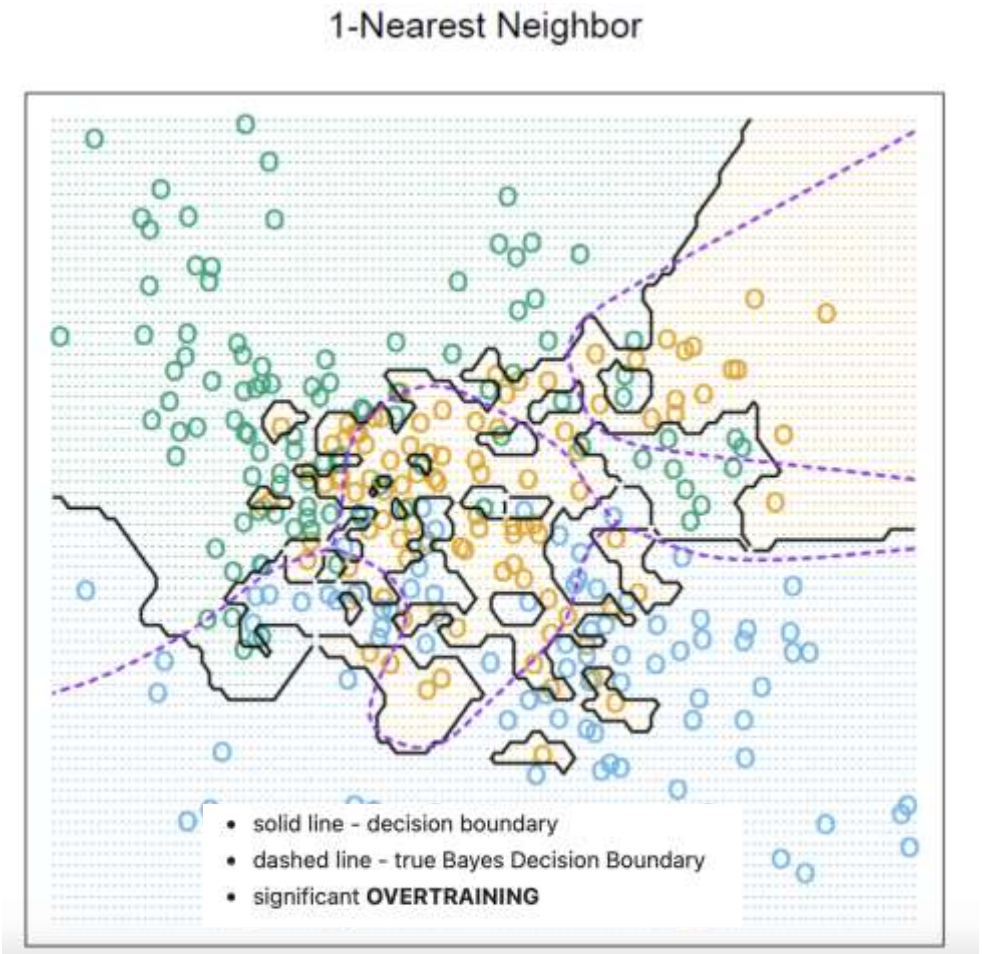
- The number of nearest neighbours is a free parameter.

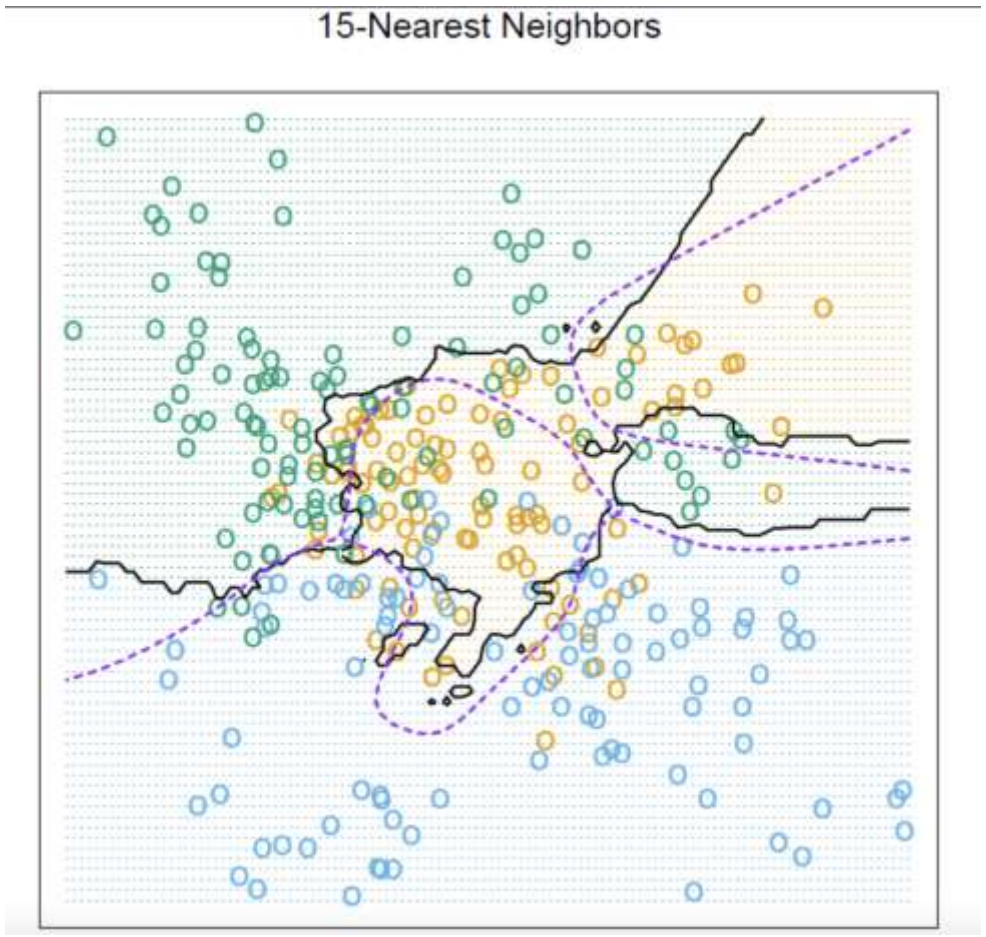


https://medium.com/data-science/k-nearest-neighbor-classifier-explained-a-visual-guide-with-code-examples-for-beginners-a3d85cad00e1

k-nearest neighbours classifier (KNN)

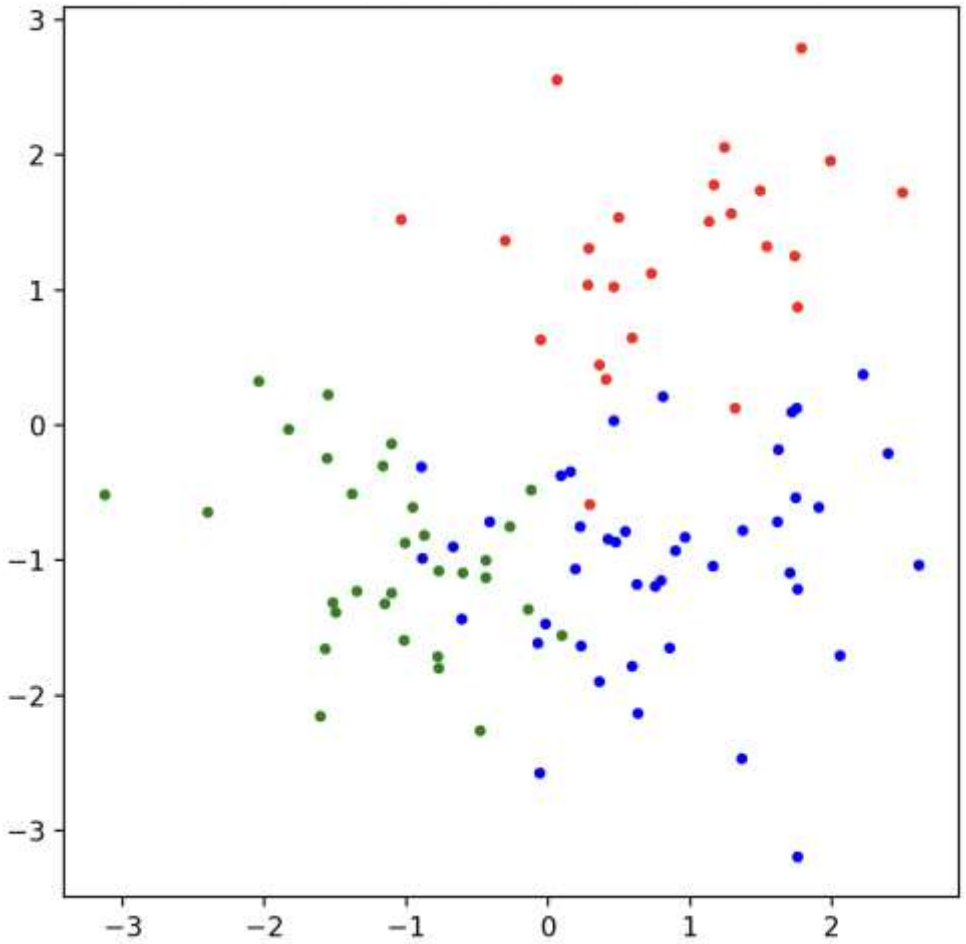
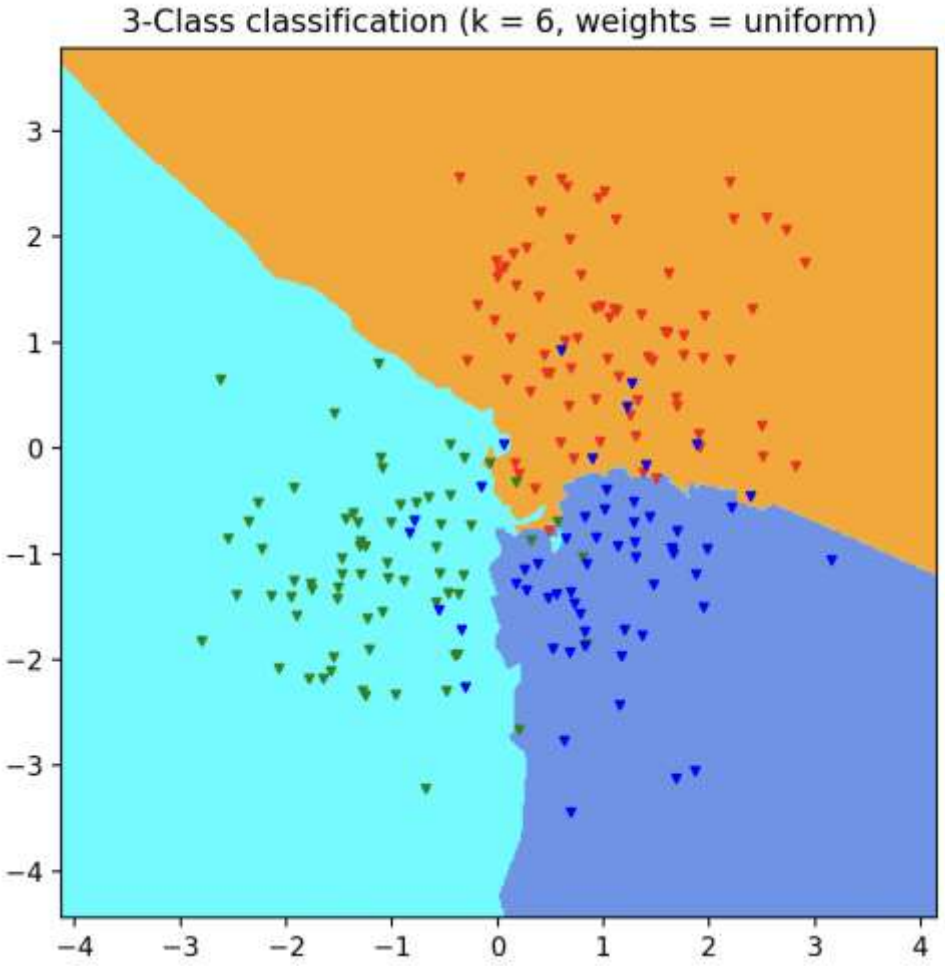
- The main parameter that can adjust the complexity of KNN is the number of neighbours *k*. The larger *k* is, the smoother the classification boundary.





kNN example

Train sample



Test sample